

CamPU: A Multi-Camera Processing Unit for Deep Learning-based 3D Spatial Computing Systems

Dongseok Im
Information & Electronics Research Institute
KAIST
Daejeon, South Korea
dsim@kaist.ac.kr

Hoi-Jun Yoo
School of Electrical Engineering
KAIST
Daejeon, South Korea
hjyoo@kaist.ac.kr

Abstract—A 3D spatial computing system that understands a surrounding environment and interacts with real-world objects has emerged with the development of deep learning technologies. A multi-camera system captures a surrounding view of a scene using multiple cameras, and a deep neural network (DNN) system extracts semantic features from multi-camera images and provides useful information to users. However, processing a multi-camera system requires massive memory accesses as the number of cameras increases while processing a DNN system can improve throughput by exploiting batch processing. This performance gap limits the overall performance of 3D spatial computing systems. To solve this problem, a multi-camera processing unit (CamPU) is proposed. CamPU exploits the inter- and intra-data reuse methods on multi-camera images, minimizing memory accesses for image projection. Moreover, the out-of-order image projection unit with cache memory is designed to increase multi-image projection throughput by avoiding redundant cache accesses and hiding the latency of high-level memory accesses. Lastly, the overlap-aware blending unit speeds up image blending by efficiently handling overlapping regions between adjacent images. The CamPU architecture is evaluated through RTL-level simulation, and the CamPU-integrated DNN platform provides a comprehensive analysis of end-to-end multi-camera deep learning-based 3D spatial systems. Finally, CamPU speeds up the overall system performance $2.9\times$ faster than an NVIDIA RTX2080Ti GPU platform.

Keywords—3D spatial computing accelerator, 360° RGB-D generation, multi-camera system, low-latency hardware architecture, image projection unit, image blending unit, cache memory

I. INTRODUCTION

Recently, 3D spatial computing systems have emerged as the development of deep learning technologies. They understand a surrounding view of a scene at a time by exploiting multiple cameras, providing immersive experiences and innovative functionalities on artificial intelligence (AI) applications. Figure 1 illustrates an example of a multi-camera deep learning-based 3D spatial computing system. AR/VR devices such as Microsoft HoloLens 2 [46], Meta Oculus Quest [9], and Apple Vision Pro [3] integrate more than four cameras for precise interaction with objects in a 3D world. Autonomous driving cars such as Tesla [5] support more than eight cameras surrounding a car to generate a 3D reconstruction map for the purpose of detecting cars and pedestrians. Even smartphones such as Apple iPhone [4] have more than two cameras for photography enhancement and taking spatial videos. Therefore, a

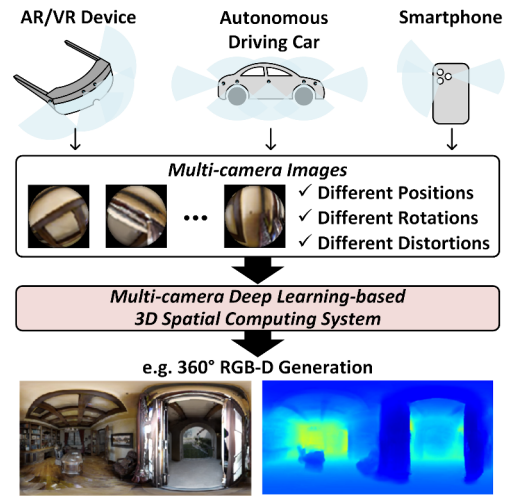


Figure 1: An example of multi-camera deep learning-based 3D spatial computing systems: 360° RGB-D generation.

multi-camera system is an essential technique for wide field-of-view (FoV) spatial computing applications. For real-time interaction, it requires low latency and energy consumption with limited resources in an edge device.

Figure 2 shows a four-stage multi-camera deep learning-based spatial computing system pipeline for wide FoV vision applications. Stage 1 is a synthesis of a unified spherical image from a number of camera images that have different positions, rotations, and distortions. It performs an inverse perspective projection (iProj) on multi-camera images, which transforms a Cartesian coordinate of each camera to a spherical coordinate in regard to their perspective views. These transformed images are stitched together, producing a unified spherical image. Stage 2 is a generation of multiple tangent images from a spherical image. Since most deep neural network (DNN) models are pre-trained with rectilinear image datasets such as ImageNet [39] and KITTI [16], Stage 2 produces virtual camera images that are the same image quality as DNN image datasets for optimal DNN performance. It applies perspective projection (Proj) on a unified spherical image and produces

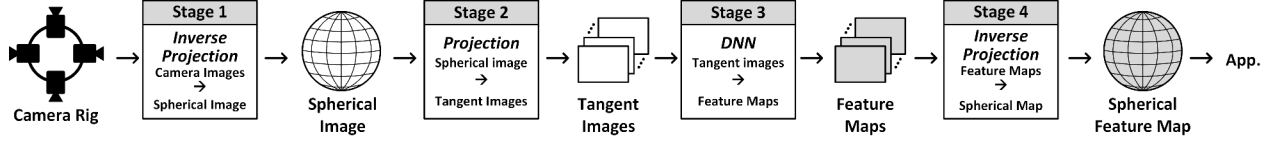


Figure 2: An overall flow of a four-stage multi-camera deep learning-based spatial computing system pipeline. Stage 1 is inverse perspective projections (iProj) on multi-camera images, Stage 2 is perspective projection (Proj) on a spherical image, Stage 3 is deep neural network (DNN) executions on each tangent image, and Stage 4 is iProj on feature maps.

tangent images according to target perspective views. Stage 3 is a DNN process on tangent images to extract their semantic features. It executes a DNN model on each tangent image and obtains semantic features such as classification, regression, and attributes. Stage 4 is a fusion of the semantic features of tangent images. It performs iProj on the DNN feature maps and stitches them to reconstruct a spherical feature map. These stages are basic flows of the multi-camera spatial computing system pipeline, and some of them could be skipped based on target applications.

The main operations of a multi-camera deep learning-based spatial computing system are image projection and DNN operations as shown in Figure 2. Image projection is a nonlinear image warping process that finds out a mapping index from a source coordinate to a destination coordinate and then applies *remap* operations on a source image with a mapping index to obtain a projected image. When a *remap* operation is performed, a large memory footprint is required to load a mapping index and corresponding source image pixels and store output pixels. On the other hand, DNN in a vision system has developed in two categories, convolutional neural network (CNN) [18], [39], [40] and vision transformer (ViT) [13]. CNN extracts visual information in receptive fields through convolutional filters while ViT understands the global context of an image by applying a self-attention mechanism [47] on a sequence of image patches. Since these DNN models have massive matrix multiplications, tremendous multiplier-and-accumulate (MAC) units are required.

However, previous spatial computing accelerators such as the GPU show inefficient implementations of multi-camera deep learning-based spatial computing systems. Although the GPU is capable of batch processing for DNN operations on multi-camera images, it under-utilizes batch processing for nonlinear image projections on them. Specifically, unlike sharing weight parameters across multi-camera images on the GPU during a DNN process, non-sharable image mapping indices cannot boost the GPU's throughput for image projection operations as the number of camera images increases. Moreover, the GPU degrades performance for image projection because of massive memory accesses of multiple intermediate data (24 MB/frame) and frequent cache misses (23% cache miss rate) caused by irregular two-dimensional patterns of the mapping index. Finally, the GPU implementation (NVIDIA RTX2080Ti [35]) of a 360° RGB-D generation system [30] shows 87.3 ms latency for image projection operations on 8

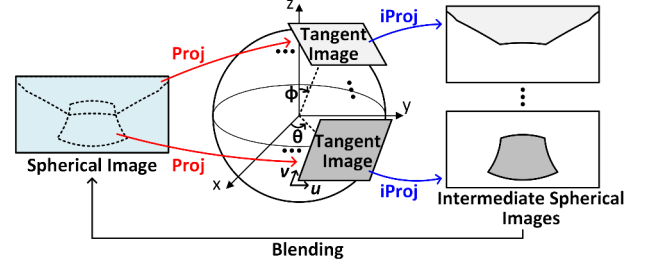


Figure 3: An illustration of perspective projection (Proj) and inverse perspective projection (iProj) operations between a spherical coordinate (θ, ϕ) and a tangent planar coordinate (u, v) .

tangent images which is $2.2\times$ slower than its DNN operations.

This paper introduces a hardware-software co-design methodology to accelerate image projection on multi-camera images. The new evaluation platform mitigates performance gaps between image projection and DNN operations, achieving $2.9\times$ faster than an NVIDIA RTX2080Ti GPU platform at a multi-camera deep learning-based spatial computing system.

II. BACKGROUND AND MOTIVATION

This section introduces basic image projection operations and their inefficient GPU implementations with design challenges. The key contributions of this work are then presented for low-latency multi-camera deep learning-based 3D spatial computing systems.

Figure 3 describes image projection operations used in spatial computing systems. Image projection operations require coordinate transformation between a spherical coordinate and a planar coordinate [42]. For a Proj operation of Stage 2, longitude (θ) and latitude (ϕ) of a spherical coordinate (θ, ϕ) are transformed to a tangent planar (u, v) with the following equation,

$$\begin{aligned} u &= \frac{\cos(\phi) \sin(\theta - \theta_c)}{\cos(c)}, \\ v &= \frac{\cos(\phi_c) \sin(\phi) - \sin(\phi_c) \cos(\phi) \cos(\theta - \theta_c)}{\cos(c)}, \\ \cos(c) &= \sin(\phi_c) \sin(\phi) + \cos(\phi_c) \cos(\phi) \cos(\theta - \theta_c) \end{aligned} \quad (1)$$

where (θ_c, ϕ_c) is a spherical coordinate of the center of a tangent planar. Using the equation 1, a spherical image is

mapped to a tangent image. To fill out all output pixels, each output pixel is calculated through inverse warping which projects four neighbor source pixels and interpolates them through bilinear interpolation.

For an iProj operation of Stage 1 and Stage 4, coordinate transformation from a tangent planar to a spherical coordinate is the following formula,

$$\begin{aligned}\theta &= \theta_c + \tan^{-1}\left(\frac{u \cdot \sin(c)}{\gamma \cos(\phi_c) \cos(c) - v \cdot \sin(\phi_c) \sin(c)}\right), \\ \phi &= \sin^{-1}(\cos(c) \sin(\phi_c) + \frac{1}{\gamma} v \cdot \sin(c) \cos(\phi_c))\end{aligned}\quad (2)$$

where $\gamma = \sqrt{u^2 + v^2}$ and $c = \tan^{-1}\gamma$. Each tangent image is mapped to a spherical image through the equation 2 with inverse warping. Additionally, projected intermediate spherical images are stitched together through image blending, generating a unified spherical image. An image blending algorithm merges overlapping pixels between adjacent intermediate spherical images with blending weights to reduce visual artifacts [37].

If multiple cameras are fixed in a multi-camera rig (and a multi-camera rig is movable in a real-world coordinate), the mapping index in regard to the equation 1 and 2 is invariant. Once the mapping index is calculated, it does not need to be updated in every frame. Therefore, the mapping index is stored in a lookup table (LUT), and a processing unit loads it and performs *remap* operations on an image for image projection. The LUT-based image projection alleviates computational costs of the equation 1 and 2 by replacing them with memory operations.

However, the LUT-based image projection limits hardware throughput because of massive memory accesses. Memory-intensive iProj operations of Stage 1 and Stage 4 require large-sized mapping indices (1 MB/image) and generate large-sized spherical images (2 MB/image) that are further stitched together through blending algorithms. Moreover, mapping indices of images show different shapes and values by latitude and longitude of a spherical coordinate based on the equation 1 and 2. As a result, processing these large amounts of inconsistent data brings about critical problems in hardware architectures as follows.

The LUT-based multi-image projection has low data reuse, causing massive memory accesses. To perform the LUT-based image projection, an image projection unit loads mapping index data from a LUT and then fetches target input pixels from memory based on a mapping index. However, since values and shapes of mapping indices are different among multi-camera images, an image projection unit cannot reuse large-sized mapping index data across multi-camera images. Moreover, an image projection unit fetches massive multi-image data and generates large-sized intermediate spherical images when it applies iProj for executions of Stage 1 and Stage 4. Therefore, as the number of cameras increases, image projection units require that amount of massive memory accesses. In specific, Figure 4 (a) describes speedup per image performance with batch processing on RTX2080Ti. The batch

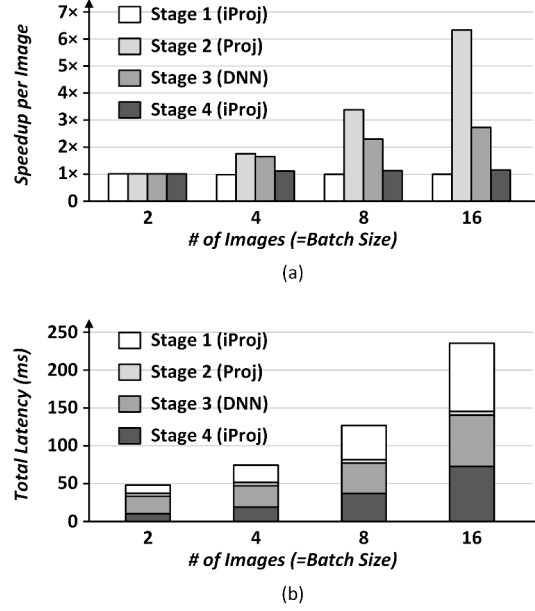


Figure 4: GPU performance of DNN and image projection operations with batch processing: (a) speedup per image and (b) total latency.

processing of Stage 3 (DNN) can increase throughput as the number of images increases by sharing weight parameters across batch inputs. Similarly, Stage 2 (Proj) for generating multiple tangent images achieves significant performance enhancement by sharing a single spherical input image across small-sized mapping indices (0.125 MB/image) corresponding to perspective views. On the other hand, multi-image projection of Stage 1 and Stage 4 (iProj) cannot enhance performance for batch processing because of non-sharable large-sized mapping indices and intermediate spherical images. Therefore, it linearly increases processing times as the number of images increases, resulting in no speedup. Moreover, Stage 1 and Stage 4 dominates the overall processing time as the number of images increases as shown in Figure 4 (b). Consequently, a new image projection unit is required to alleviate memory accesses for multi-image projection.

Massive *remap* operations in image projection bring about redundant instruction issues and cache memory accesses. A *remap* operation is a geometrical transformation function that loads an input pixel from memory and stores it in another address of memory. It brings about irregular memory accesses for image projection caused by a nonlinear image warping process based on the equation 1 and 2. Moreover, inverse warping performs four times memory-intensive *remap* operations to get an output pixel. It fills out all output pixels by loading their corresponding four neighbor input pixels, increasing the number of memory accesses by four times. To minimize the latency of memory accesses, high-speed cache memory is adopted that stores frequently accessed data. An

image projection unit with 4 KB 2-way set associative cache memory improves $1.4\times$ throughput of inverse warping by exploiting a spatial similarity of neighbor pixels (showing 6% cache miss rate). However, an in-order image projection unit with cache memory is still inefficient in executing quadruple *remap* operations of inverse warping. It takes four times the latency to issue quadruple instructions and repetitively accesses the same cache memory address where target pixels are positioned in the same cache line. In specific, the latency of instruction issues and cache memory access accounts for $2.1\times$ higher than the cache miss latency even though a cache miss is $8\times$ slower than their processes. Therefore, an efficient image projection unit with a cache memory system is necessary to alleviate the challenges of *remap* operations.

Processing image blending on non-rectangular projected outputs is incompatible with the conventional memory system. The shape of iProj outputs is non-rectangular as shown in Figure 3. This is because output pixels are mapped from a tangent image within an inner pixel condition $0 < u < Width$ and $0 < v < Height$ based on the equation 2. Therefore, the conventional memory system which utilizes rectangular memory blocks deteriorates the hardware performance to perform image blending across non-rectangular intermediate spherical images. The GPU could accelerate the non-rectangular image blending by performing image projection with expanded mapping indices. It expands different shapes of mapping indices to the same-sized and full-sized rectangular ones with invalid maps and applies image projection with them. Then, their rectangular projected outputs are merged by masking invalid regions. This approach allows the GPU to perform parallel computing but wastes massive redundant memory accesses (88.3% out of total data) caused by invalid regions. Therefore, image blending following image projection with invalid maps causes numerous memory footprints caused by redundant data and is a burden for parallel processing in resource-limited hardware platforms.

To solve these problems, a multi-camera processing unit (CamPU) is newly introduced.

- Inter-data and intra-data reuse methods on multi-camera images are proposed to alleviate memory accesses of the LUT-based image projection. The inter-data reuse method exploits shape similarity among mapping indices of latitude-aligned images, which benefits in sharing mapping index across them. Moreover, the intra-data reuse method exploits value similarity between adjacent mapping index elements which processes small-sized differential mapping indices. As a result, exploitation of inter- and intra-data reuse saves the LUT footprint and bandwidth during the LUT-based image projection.
- The out-of-order image projection unit with cache memory is proposed for high throughput *remap* operations in image projection. The load OP unit dynamically schedules and fuses memory *load* operations to access target pixels allocated in the same cache line. Then, the out-of-order memory load execution unit executes fused *load* operations and writes back to destination registers simul-

taneously, reducing the number of instruction issues and cache memory accesses. Moreover, the out-of-order execution hides the latency of high-level memory accesses. Finally, the pipelined out-of-order image projection unit significantly reduces the overall image projection latency.

- The overlap-aware blending unit is proposed for high throughput of image blending. It merges rectangular projected outputs having minimum invalid regions and offsets an output coordinate by the offset controller. Moreover, the overlap-aware blending unit minimizes redundant memory footprints by handling overlapping regions between adjacent images. Consequently, it alleviates memory accesses caused by non-rectangular projected images in the memory system.
- RTL-level simulation of the CamPU architecture provides a cycle-accurate architectural analysis, and the CamPU-integrated DNN platform is designed for a comprehensive evaluation of a multi-camera deep learning-based spatial computing system. The evaluation results demonstrate the critical role of CamPU, showing low latency of the end-to-end system performance with minimal hardware costs.

III. CAMPU ARCHITECTURE

A. Overall Architecture and Dataflow

Figure 5 describes the overall CamPU architecture and its dataflow that performs LUT-based image projection. CamPU consists of four CamPU cores each of which consists of the index decoder unit, the image projection unit, 2 KB of projected output buffer, and the blending unit. The index decode unit applies inter- and intra-data reuse methods that achieve significant benefits from sharing mapping index and memory footprint reduction (Section III-B). The mapping index LUT stores differential mapping index ($\Delta u, \Delta v$) obtained through pre-computations of the equation 2 (intra-data reuse). Then, the index recovery unit accesses the mapping index LUT corresponding to the target address (θ_n, ϕ_n) , and recovers the original mapping index (u_k, v_k) by adding differential mapping index $(\Delta u_k, \Delta v_k)$ to the previously recovered one (u_{k-1}, v_{k-1}) . Moreover, the mapping index is shared across the latitude-aligned multiple images ($I_0(u_k, v_k), I_1(u_k, v_k), I_2(u_k, v_k), I_3(u_k, v_k)$) that increases throughput of multi-image projection (inter-data reuse). For a single image projection, CamPU only exploits intra-data reuse on the mapping index and still reduces the size of the mapping index.

After obtaining the mapping index, the image projection unit performs *remap* operations ($I(u, v) \rightarrow O(\theta, \phi)$). It adopts cache memory that exploits frequently accessed image pixels during image projection, which reduces the latency of memory accesses (Section III-C). Thanks to the inter-data reuse, the memory load execution unit loads multi-image pixels aligned in the same cache line at a time, increasing the throughput of multi-image projection. For image projection on a single image, adjacent pixels of an image are aligned in the cache line, which increases the number of cache hits. The image projection unit also supports bilinear interpolation for inverse warping. The final projected outputs ($O_0(\theta_n, \phi_n), O_1(\theta_n, \phi_n)$,

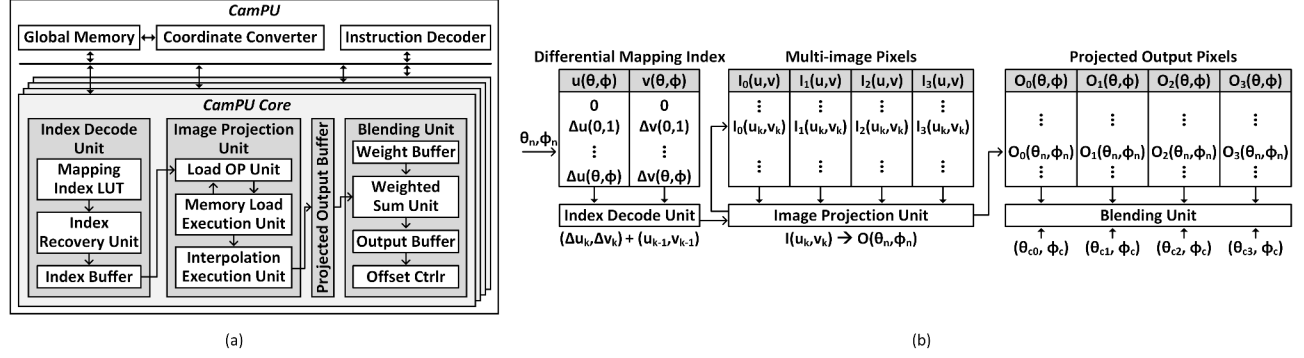


Figure 5: (a) An overall CamPU architecture and (b) its dataflow.

$O_2(\theta_n, \phi_n), O_3(\theta_n, \phi_n)$ are stored into the projected output buffer and are processed by the blending unit for image blending. When CamPU only performs image projection without image blending, it deactivates the blending unit and transfers projected outputs to the global memory as final outputs.

The blending unit blends the projected outputs to generate a unified spherical image. It applies the overlap-aware rectangular image blending method to reduce redundant memory footprints (Section III-D). The blending unit fetches projected output pixels and corresponding blending weights, and performs weighted summation. The blending outputs are offset through the offset controller by adding each center index of spherical images $(\theta_{c0}, \phi_c), (\theta_{c1}, \phi_c), (\theta_{c2}, \phi_c), (\theta_{c3}, \phi_c)$. The blending outputs are finally stored in the global memory.

CamPU integrates 256 KB of global memory to load and store intermediate data across CamPU cores, taking 8 cycles of latency on average. The coordinate converter unit computes the equation 1 and 2. It exploits a piecewise linear approximation of trigonometric functions through single instruction multiple data (SIMD) units and direct global memory access without occupying the interconnect network. Once the mapping index is calculated, the coordinate converter unit is deactivated until an update is needed. The instruction decoder receives instructions from the CPU and issues them to target hardware units. The interconnect network connects all of the hardware units providing sufficient bandwidth.

B. Exploitation of Inter-data and Intra-data Reuse Methods on Multi-camera Images

Figure 6 shows the inter- and intra-data reuse methods on mapping indices to alleviate memory accesses during the LUT-based image projection. Multiple images (I_0, I_1, I_2, I_3) at the same latitude show the same shape of mapping indices (spherical rectangles as shown in the figure) each other whose values only differ by a center index (θ_c) on a θ -coordinate based on the equation 2. Therefore, CamPU performs image projection on latitude-aligned multi-image with a shared mapping index (inter-data reuse), and their projected outputs (O_0, O_1, O_2, O_3) are offset by different values of mapping indices $(\theta_{c0}, \theta_{c1}, \theta_{c2}, \theta_{c3})$ in a spherical coordinate. Specifically, the index decode

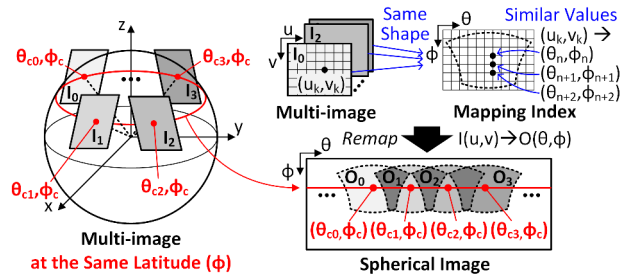


Figure 6: Inter-data reuse (shape similarity) and intra-data reuse (value similarity) methods during iProj on latitude aligned multi-camera images.

unit stores a single mapping index corresponding to four images in the mapping index LUT, saving 75% of the LUT footprint and LUT bandwidth using the inter-data reuse. Moreover, exploiting latitude-aligned multi-image benefits from efficient *remap* operations and image blending operations. The image projection unit performs *remap* operations on latitude-aligned multi-image simultaneously by aligning multi-image pixels in a cache line. Similarly, the blending unit efficiently merges the same shapes of projected outputs in parallel. Detailed benefits will be explained in Section III-C and Section III-D, respectively.

Some applications could not use latitude-aligned multi-image during Stage 1 because of a constraint camera rig. In that case, CamPU exploits the value similarity of mapping index elements (intra-data reuse) to save memory accesses. The intra-data reuse differentiates similar values of adjacent elements in a mapping index during a single image projection. As shown in an example of Figure 6, the adjacent elements in a mapping index, $(\theta_n, \phi_n), (\theta_{n+1}, \phi_{n+1}), (\theta_{n+2}, \phi_{n+2})$, indicate the same input pixel at (u_k, v_k) and can be encoded to all $(0, 0)$ differential elements with a reference mapping index of (u_k, v_k) . By exploiting intra-data reuse, the bit-precision of a mapping index is reduced from 8-bit to 2-bit for image projection on 256×256 sized images. Then, the index decode unit recovers differentiating mapping indices for *remap*

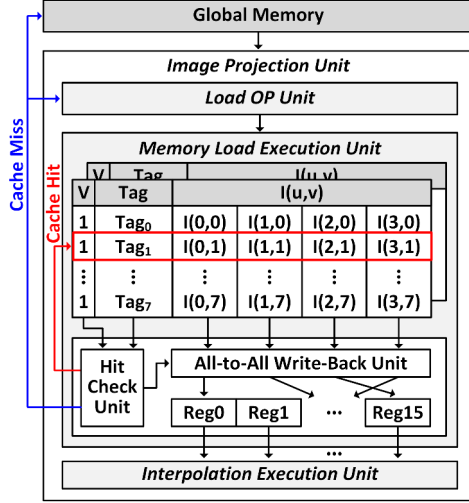


Figure 7: An architecture of the proposed image projection unit for low-latency *remap* operations.

operations. Therefore, the intra-data reuse method compresses a mapping index by 75%. Finally, a mapping index can be reduced by 94.4% by exploiting both inter-data and intra-data reuse methods for iProj operations on 256×256 sized 18 images at 4 different latitudes and 3, 6, 6, and 3 different longitudes.

C. Out-of-order Image Projection Units with Cache Memory

A *remap* operation is a generic memory *load* operation with a mapping index. However, massive irregular *remap* operations in image projection slow down the overall processing times. Moreover, inverse warping performs four times *remap* operations to interpolate four neighbor pixels and requires four times memory accesses. Although parallel processing is a key solution for throughput enhancement, naive implementations bring about inefficient memory *load* operations such as duplicate instruction fetches and memory bank conflicts.

Figure 7 illustrates the proposed image projection unit that alleviates massive and irregular memory *load* operations in *remap* operations. The load OP unit fetches *remap* instructions from the CPU, decodes them, and issues *load* operations to the memory load execution unit. The memory load execution unit executes memory *load* operations based on a mapping index. It exploits high-speed cache memory to minimize expensive high-level memory accesses. Cache memory consists of a 4KB 2-way set-associative cache, each of which is used for either cache hit or cache miss cases. It only supports a memory *load* operation, which reduces hardware costs for cache management. Cache memory holds frequently accessed input pixels with their tag and valid bits, and a cache line contains four 16-bit pixel values. The hit check unit decodes *load* operations and compares their target addresses to tag and valid bits to decide cache hit status. After loading a cache line from cache memory into temporal registers, the write-back unit writes target pixels in a cache line to corresponding write-back

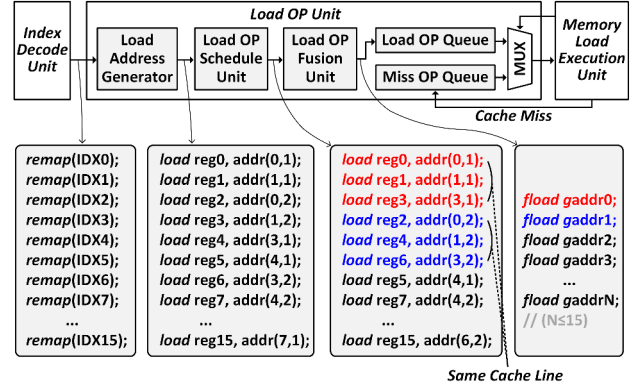


Figure 8: An architecture of the load OP unit with dynamic instruction scheduling.

registers through an all-to-all connection, and a maximum of sixteen write-back registers (Reg0–Reg15) could be written at once. If a cache miss occurs, the image projection unit accesses high-level global memory and stores the issued *load* operations of a cache miss to the miss OP queue that would be used when the high-level memory data is available. Finally, the memory load execution unit propagates remapping outputs to the interpolation execution unit for bilinear interpolation.

To reduce the number of instruction issues and cache accesses, the load OP unit with dynamic instruction scheduling is proposed as shown in Figure 8. The load OP unit fetches sixteen *remap* instructions with recovered mapping indices from the index decode unit. Then, the load address generator decodes them and produces a group of memory *load* instructions with target memory addresses. The load OP schedule unit reorders *load* instructions whose memory addresses are in the same cache line. Then, the load OP fusion unit fuses the cache-line-aligned *load* instructions into a single *fload* instruction. As illustrated in an example of Figure 8, when a cache line holds four subsequent pixels whose addresses are *addr*(0, 1), *addr*(1, 1), *addr*(2, 1), and *addr*(3, 1), *load* instructions addressing *addr*(0, 1), *addr*(1, 1), and *addr*(3, 1) are fused into a single *fload* instruction indicating a group address (*gaddr*0). Finally, the *fload* instructions are stored in the load OP queue and issued to the memory load execution unit in a sequence. By fusing memory *load* operations, the number of instruction issues and cache memory accesses are significantly reduced. In iProj with an inverse warping process, memory *load* operations are reduced by 72.7%. As a result, the load OP unit remarkably reduces instruction issues and cache memory accesses through a fusion of *load* operations.

To efficiently execute the *fload* operations, the image projection unit supports out-of-order memory *load* executions. The load OP unit issues a *fload* instruction stream to the memory load execution unit. The memory load execution unit loads the target cache line from cache memory, and the all-to-all write-back unit writes back target pixels within the cache line to corresponding registers simultaneously. If

a cache miss occurs, the issued *fload* operation is pushed to the miss OP queue in the load OP unit and requests a corresponding memory block to the high-level global memory. When the memory block is ready, the memory load execution unit fetches the *fload* operation from the miss OP queue and executes the *fload* operation on the memory block. During cache miss latency, the memory load execution unit continues to fetch the next *fload* instruction from either the load OP queue or the miss OP queue and then processes it not relying on the cache hit status. This parallel process can hide the cache miss latency of memory *load* executions. After finishing sixteen *remap* operations, the memory load execution unit commits the write-back registers and passes them to the interpolation execution unit. This out-of-order operation significantly reduces the latency of image projection by 74.9% by reducing the number of cache accesses and hiding a cache miss latency.

The interpolation execution unit performs weighted summation operations on neighbor pixels obtained by the memory load execution unit. The interpolation execution unit directly accesses the write-back registers of the memory load execution unit and performs bilinear interpolation. Interpolation outputs are stored in the projected output buffer and applied to following image blending operations.

The pipelined architecture of the load OP unit, the memory load execution unit, and the interpolation execution unit efficiently execute the memory-intensive image projection and bilinear interpolation by hiding each latency. Therefore, the out-of-order image projection unit improves the throughput of iProj with inverse warping, achieving $3.99\times$ higher throughput than the in-order image projection unit for a single image projection. Moreover, it efficiently accelerates multi-camera image projections on latitude-aligned images. Thanks to the inter-data reuse among the latitude-aligned mapping indices, the image projection unit loads the shared mapping index and performs *remap* operations on multi-camera images at a time. A cache line consists of four pixels of four images, and the memory load execution unit simultaneously accesses four pixels for multi-image projection. As a result, the out-of-order image projection unit enhances the throughput of multi-image projection on four images by $3.17\times$ higher than that of single image projection.

D. Overlap-aware Blending Unit with Rectangular Projected Outputs

The overlap-aware blending unit is proposed to efficiently execute image blending. Unlike processing full-sized intermediate spherical images, CamPU performs image projection with a small-sized rectangular mapping index that has minimal invalid regions with an indication of the center coordinate (θ_c, ϕ_c) of each projected output. Although invalid regions still exist in projected outputs, processing image blending on rectangular projected outputs is compatible with the conventional memory system and allows a blending unit to efficiently handle overlapping regions among them. Consequently, CamPU exploits a rectangular mapping index for image projection and

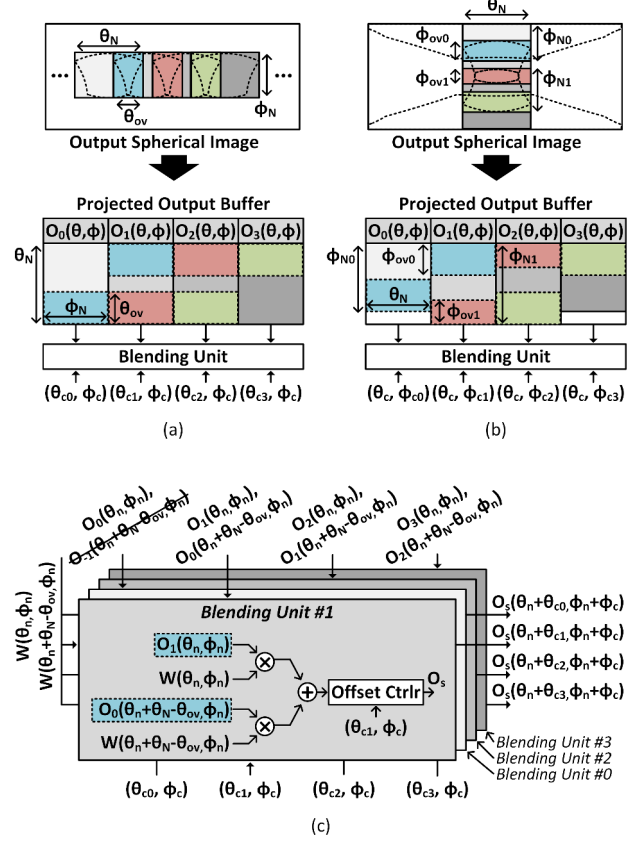


Figure 9: A concept of the overlap-aware image blending unit with rectangular projected outputs: (a) image blending on latitude-aligned projected output images, (b) image blending on longitude-aligned projected output images, and (c) dataflow of the overlap-aware blending unit.

then applies image blending on rectangular projected outputs, reducing intermediate data size by 81.9% compared to the full-sized image process on the GPU.

As shown in Figure 9 (a), the overlap-aware blending unit aggregates the latitude-aligned rectangular projected images. After image projections on latitude-aligned images, their projected outputs have the same rectangular shapes $(\theta_N \times \phi_N)$, and their overlapping regions are also the same $(\theta_{ov} \times \phi_N)$ among the projected outputs (O_0, O_1, O_2, O_3) . Moreover, overlapping regions occur symmetrically between adjacent projected images. Therefore, the projected output buffer stores the same-sized projected outputs, and overlapping regions are allocated in the buffer symmetrically. Similarly, the overlap-aware blending unit processes longitude-aligned rectangular projected images as shown in Figure 9 (b). Unlike a latitude-aligned process, the shapes of the mapping indices are different by latitude. Therefore, projected spherical images are cropped to have the same width (θ_N) , and then the cropped projected images are stitched together through the blending unit. To efficiently blend all projected images, the blending unit stitches

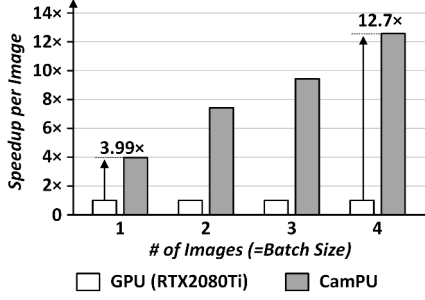


Figure 10: CamPU performance of image projection and blending operations with batch processing.

all latitude-aligned images in each longitude at first and then blends the stitching outputs across different longitudes.

Figure 9 (c) describes a dataflow of the overlap-aware blending unit. The blending unit aggregates projected outputs (O_0, O_1, O_2, O_3) and stitches them with the blending weights (W) to generate a unified spherical image (O_s). The blending unit loads projected outputs from the projected output buffer and blending weights from the weight buffer, and it applies weighted summation. By exploiting symmetric overlapping regions, the overlap-aware blending unit loads a pair of projected images, $O_1(\theta_n, \phi_n)$ and $O_0(\theta_n + \theta_N - \theta_{ov}, \phi_n)$, from the projected output buffer and performs weighted summation with corresponding blending weights $W(\theta_n, \phi_n)$ and $W(\theta_n + \theta_N - \theta_{ov}, \phi_n)$. Final output (O_s) is offset by the offset controller before propagating to the global memory. Finally, the overlap-aware blending unit achieves 53.1% memory access reduction when image blending on 18 images at 4 different latitudes and 3, 6, 6, and 3 different longitudes.

IV. ARCHITECTURE EVALUATIONS

A. Simulation Environment of Architecture Evaluations

CamPU is designed in a gate-level synthesis by the Synopsys Design Compiler with 28 nm technology and 500 MHz of clock frequency. Its power is measured through the Synopsys PrimeTime PX. CamPU occupies 0.54 mm^2 of area and consumes 12.9 mW of power on average. The CamPU architecture is evaluated through RTL-level simulation running multi-camera systems that have 80-120° field-of-view (FoV) of 256×256 sized camera images and stitching their DNN outputs on a spherical coordinate. The baseline architecture consists of the in-order image projection unit and blending unit; the baseline's image projection unit adopts cache memory with the in-order memory load execution, and its blending unit processes full-sized intermediate spherical images without overlap-aware rectangular image blending.

B. Performance on Different Numbers of Cameras

Figure 10 shows the speedup per image performance on image projection with inverse warping and image blending in different numbers of camera images. A GPU (NVIDIA RTX2080Ti [35]) architecture cannot increase throughput due

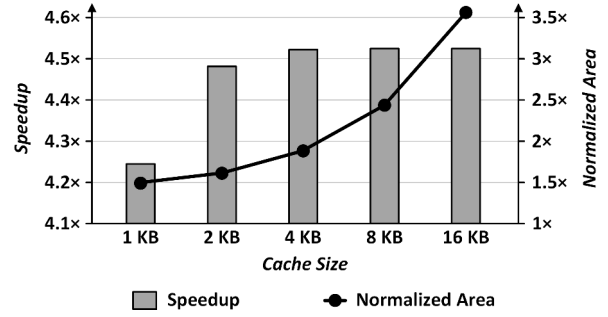


Figure 11: Speedup and area overheads of the out-of-order image projection unit by cache size.

to low data reuse of multi-image projections and inefficient image blending operations. On the other hand, CamPU improves throughput as the number of camera images increases by exploiting latitude-aligned images. By sharing a mapping index across latitude-aligned images, the image projection unit boosts *remap* operations with reduced LUT bandwidth, and the blending unit speeds up overlap-aware blending operations on symmetric overlapping regions. Moreover, CamPU can accelerate image projection on a single-camera image (1 batch size) through intra-data reuse on a mapping index and the out-of-order image projection. Finally, CamPU achieves $3.99\times$ and $12.7\times$ higher speedup per image than the GPU at a single-camera image and four-camera images, respectively.

C. Performance and Area Overheads by Different Cache Memory Sizes

Figure 11 shows the speedup and area overhead of the out-of-order image projection unit by cache memory size. By adopting 1 KB of cache memory, the image projection unit achieves $4.3\times$ throughput with $1.5\times$ area overhead compared to no cache memory. Increasing cache sizes results in a low cache miss rate so that the throughput of image projection is enhanced as a cache memory size increases. However, throughput enhancement becomes saturated at larger than 4 KB of cache memory size. Additionally, a large cache size causes a large area overhead of the image projection unit. The image projection unit with 16 KB of cache memory size shows a $3.6\times$ area overhead. By considering both performance and area overheads, the out-of-order image projection unit adopts 4 KB of cache memory size and achieves $4.5\times$ speedup with $1.9\times$ area overhead.

D. Ablation Study of CamPU

Figure 12 shows the ablation study for image projection and blending. The baseline architecture integrates the in-order image projection unit with the cache memory and the blending unit processing full-sized intermediate spherical images. The version 1 architecture adopts out-of-order image projection from the baseline architecture and increases the overall throughput by $3.0\times$. The version 2 architecture integrates the overlap-aware blending unit for accelerating image blending,

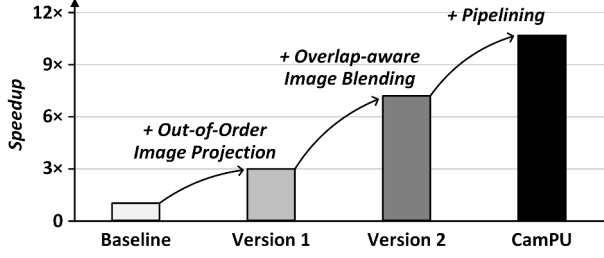


Figure 12: Ablation study of CamPU.

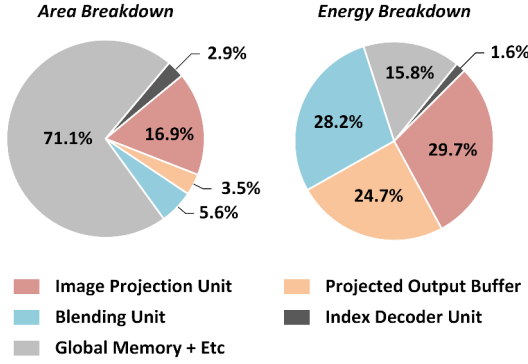


Figure 13: Area and energy consumption breakdowns of CamPU.

improving overall performance by $2.4\times$ higher than the version 1 architecture. The last version architecture (CamPU) exploits the pipelined image projection and overlap-aware blending units and hides the latency between them, achieving $1.5\times$ throughput enhancement. Finally, CamPU accomplishes $10.7\times$ speedups compared to the baseline architecture.

E. Area and Power Breakdown of CamPU

Figure 13 illustrates area and energy consumption breakdowns of the CamPU architecture. The image projection unit takes 16.9% of the total area, the blending unit accounts for 5.6%, the projected output buffer shows 3.5%, and the index decoder unit takes 2.9%. Global memory with other components (the coordinate converter, the instruction decoder, etc) takes 71.1% of the total area, which is dominant due to its large-sized SRAM memory. On the other hand, the energy consumption is dominated by the image projection unit, consuming 29.7% of the total energy because of its complex datapath for the out-of-execution. Similarly, the blending unit takes 28.2% of the total energy consumption because of massive memory accesses by pairs of image pixels and blending weights. The projected output buffer takes 24.7%, global memory with other components takes 15.8%, and the index decoder unit accounts for 1.6%.

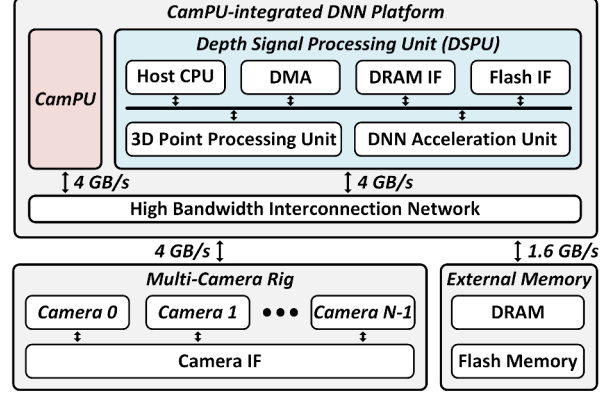


Figure 14: An overall architecture of the CamPU-integrated DNN evaluation platform for multi-camera deep learning-based 3D spatial computing systems.

V. SYSTEM EVALUATIONS

A. Simulation Environment of System Evaluations

Figure 14 describes the CamPU-integrated DNN platform for a multi-camera deep learning-based 3D spatial computing system. The platform has been upgraded from a previous 3D spatial computing platform [20] to support a multi-camera system. A new platform consists of CamPU and a 3D depth signal processing unit (DSPU) [21], attaching a multi-camera rig and external memory. The DSPU is a host device integrating the RISC-V CPU, external low-power DRAM [12] and flash memory [11] interfaces, a 3D point processing unit, and a DNN acceleration unit. The RISC-V CPU controls all hardware components including CamPU, a multi-camera rig, and external memory. The CPU sends instruction streams to CamPU through the interconnection network, and the CamPU's instruction decoder stores them in an instruction buffer. Then, CamPU sequentially decodes instructions from the buffer and activates the target hardware units. The DSPU performs high-speed deep learning-based image-to-image tasks such as monocular depth estimation [7], [17] and image segmentation [38] on a camera image and additionally extracts 3D perception such as a 3D bounding box [36] by activating a 3D point processing unit and a DNN acceleration unit. The DSPU is designed in 28 nm technology occupying 12.96 mm^2 of area and consuming 766.0 mW under 500 MHz of operating frequency. The packet-based interconnection network connects CamPU, the DSPU, and a multi-camera rig offering high bandwidth (4 GB/s). The external low-power DRAM provides 1.6 GB/s bandwidth and an average latency of 70 cycles.

The CamPU-integrated DNN platform executes 360° RGB-D generation and image segmentation systems, processing 256×256 and 320×240 sized tangent images, respectively. A multi-camera rig samples images from multiple cameras and supplies multi-image batches to the CamPU-integrated DNN platform through the high bandwidth interconnection

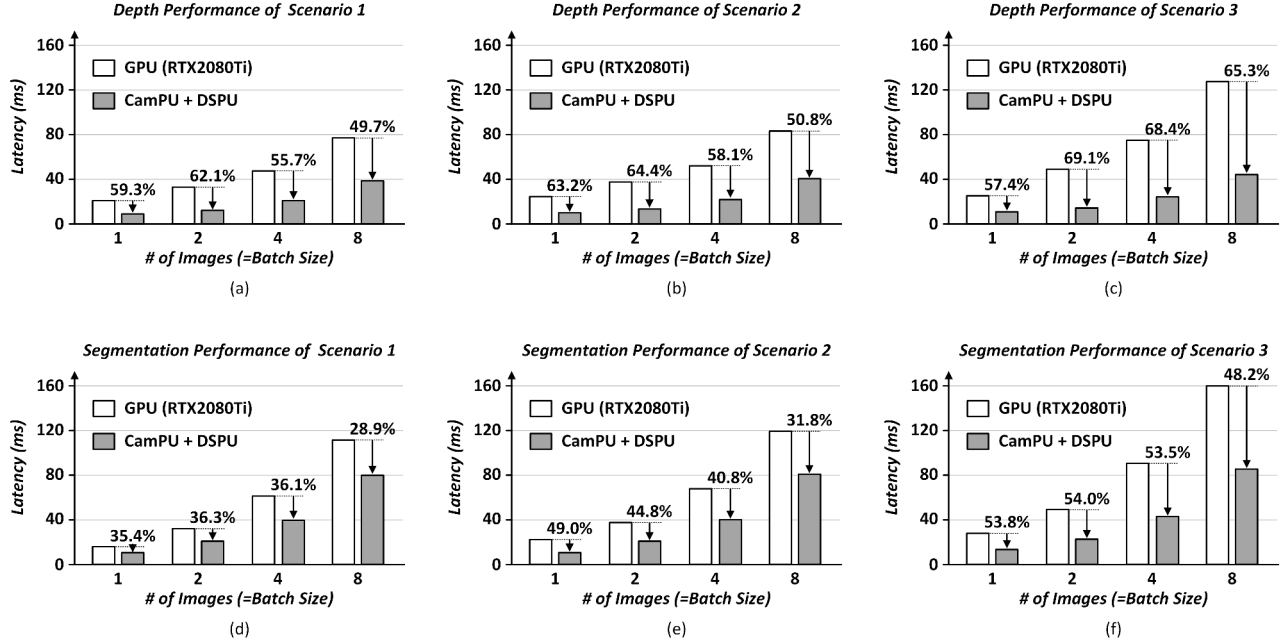


Figure 15: An end-to-end latency of the GPU platform and CamPU-integrated DNN platform running multi-camera deep learning-based 3D spatial computing systems by different numbers of cameras: (a) monocular depth estimation with Scenario 1, (b) monocular depth estimation with Scenario 2, (c) monocular depth estimation with Scenario 3, (d) image segmentation with Scenario 1, (e) image segmentation with Scenario 2, and (f) image segmentation with Scenario 3. *Scenario 1: Stage 3 (DNN) + Stage 4 (iProj), Scenario 2: Stage 2 (Proj) + Stage 3 (DNN) + Stage 4 (iProj), and Scenario 3: Stage 1 (iProj) + Stage 2 (Proj) + Stage 3 (DNN) + Stage 4 (iProj).

network. CamPU receives multi-camera images and stitches them to generate a unified spherical RGB image. Then, it produces the same quality and different perspective views of tangent images for DNN processing. The DSPU performs deep learning-based image-to-image tasks on tangent images as a multi-batch parallel process. After obtaining deep learning outputs from the DSPU, CamPU stitches them through image projection and image blending and finally generates a 360° deep learning output.

B. Performance Comparisons under Various Stages of Multi-camera Deep Learning-based 3D Spatial Computing Systems

Figure 15 describes an end-to-end latency of the GPU platform and the CamPU-integrated DNN platform evaluated in three scenarios about deep learning-based monocular depth estimation [17] (Figure 15 (a), (b), and (c)) and image segmentation [38] (Figure 15 (d), (e), and (f)) extended to a multi-camera system. In Scenario 1, the platform processes multi-camera images directly, that are horizontally aligned in a multi-camera rig, with the processes of Stage 3 (DNN) and Stage 4 (iProj). Scenario 2 is that the platform receives a spherical image from commercial 360° cameras [22], [29] and performs Stage 2 (Proj), Stage 3 (DNN), and Stage 4 (iProj) on it. Scenario 3 is that the platform runs all four stages with arbitrarily positioned cameras. Each scenario is tested under different numbers of cameras, and the performance of a single

camera (1 batch size) is also reported as processing a distorted wide FoV image like a fisheye lens along with a DNN model pre-trained with rectilinear image datasets.

NVIDIA RTX2080Ti [35] is a high-performance graphic processing unit (GPU) integrating ray-tracing cores for realistic 3D graphics and CUDA/Tensor cores for parallel computations, accomplishing high-end 3D spatial computing systems. With plentiful hardware resources, the GPU platform accelerates DNN, image projection, and image blending operations under a thermal design power (TDP) of 250 W. On the other hand, the CamPU-integrated DNN platform is a heterogeneous architecture; low-power CamPU (12.9 mW) accelerates image projection and blending operations and the low-power DSPU (766.0 mW) executes DNN operations.

In 256×256 sized monocular depth estimation executions of Scenario 1 as shown in Figure 15 (a), the GPU platform slows down the overall system latency due to non-sharable computations and massive memory accesses of image projection and blending. In contrast, the CamPU-integrated DNN platform significantly boosts the overall system performance by accelerating image projection and blending on multi-image. Finally, the CamPU-integrated DNN platform achieves 59.3%, 62.1%, 55.7%, and 49.7% latency reductions at 1, 2, 4, and 8 numbers of multi-image processing compared to the GPU platform, respectively.

In Scenario 2, the GPU platform accelerates an additional execution of Stage 2 through multi-batch processing with sufficient hardware resources, showing slightly higher latency than Scenario 1 as illustrated in Figure 15 (b). Similarly, CamPU boosts an execution of Stage 2 through LUT-based image projection so that its latency overhead is minimal compared to the execution of Scenario 1. As a result, the CamPU-integrated platform reduces latency by 63.2%, 64.4%, 58.1%, and 50.8% at 1, 2, 4, and 8 numbers of images compared to the GPU platform, respectively.

Figure 15 (c) describes the depth performance of the platforms running Scenario 3. Heavy iProj operations of Stage 1 and Stage 4 deteriorate the overall system performance of the GPU platform, resulting in the slowest performance among the scenarios. However, the CamPU-integrated DNN platform efficiently accelerates dominant iProj operations, reducing latency by 57.4%, 69.1%, 68.4%, and 65.3% at 1, 2, 4, and 8 numbers of image batch sizes compared to the GPU platform, respectively, whose amounts are the largest reductions among the scenarios.

Figure 15 (d), (e), and (f) illustrate 320×240 sized image segmentation executions of Scenario 1, 2, and 3, respectively. Relatively large image and DNN sizes of the image segmentation task limit latency reduction compared to the monocular depth estimation task because of a lack of hardware resources for multi-batch DNN executions in the CamPU-integrated platform compared to the GPU platform. Nonetheless, the CamPU-integrated DNN platform surpasses the GPU platform, showing 35.4%, 36.3%, 36.1%, and 28.9% at 1, 2, 4, and 8 numbers of images in Scenario 1, 49.0%, 44.8%, 40.8%, and 31.8% in Scenario 2, and 53.8%, 54.0%, 53.5%, and 48.2% in Scenario 3, respectively. Finally, the CamPU-integrated DNN platform accomplishes the lowest system latency in multi-camera deep learning-based spatial computing systems.

C. Latency Breakdown among Various Platforms running 360° Spatial Computing Systems

Figure 16 illustrates the system latency breakdown of various platforms running four-stage 360° spatial computing systems with 18 multi-camera images at 4 different latitudes and 3, 6, 6, and 3 different longitudes. Figure 16 (a) shows the latency of 360° RGB-D generation benchmark [30] with 256×256 sized tangent images. The RTX2080Ti platform executes all stages of the system pipeline, taking 270.1 ms of the system latency. However, the latency of image projection and image blending on multi-camera images accounts for 69.8% of the total latency. Therefore, the CamPU-integrated GPU platform which assumes an integration of dedicated CamPU into the GPU architecture offloads image projection and image blending tasks to CamPU, remarkably reducing the overall system latency by 63.1%. The CamPU-integrated RTX2080Ti platform achieves under 100 ms of end-to-end processing time. On the other hand, the DSPU accelerates DNN tasks in much lower latency (76.1 ms) and lower power consumption (766.0 mW) than RTX2080Ti (82.0 ms of latency and a TDP of 250 W). However, the DSPU platform

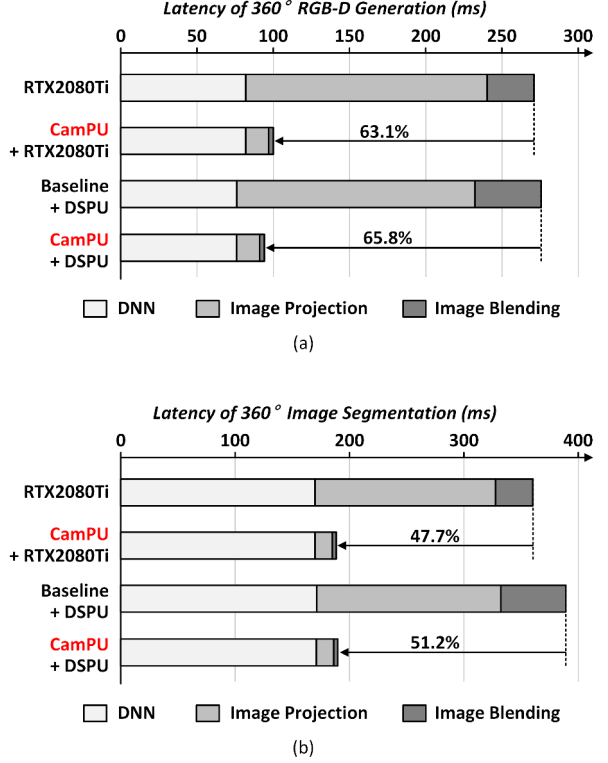


Figure 16: System latency breakdown of various platforms running four-stage 360° spatial computing systems with 18 multi-camera images: (a) 360° RGB-D generation and (b) 360° image segmentation.

requires an additional accelerator for a multi-camera system. Slow multi-image projection and blending of the baseline architecture, which performs the in-order image projection and image blending on full-sized intermediate images, is a bottleneck in the overall system, and the baseline + DSPU platform shows 275.6 ms of processing time. On the other hand, CamPU significantly reduces a multi-camera system by 65.8% with minimal area occupancy and power consumption. Consequently, the CamPU + DSPU platform achieves the lowest latency (94.1 ms) among the 3D spatial computing platforms, which is $2.9\times$ faster than the RTX2080Ti platform.

Figure 16 (b) describes the system latency breakdown of 360° image segmentation benchmark [38] with 320×240 sized tangent images. Although large image and DNN sizes of the image segmentation system take more latency compared to the RGB-D generation system, the CamPU only has 3% latency overheads of image projection and image blending in regard to processing 17% increased image pixels. Finally, the CamPU-integrated platforms achieve under 200 ms of end-to-end processing time, reducing the system latency by 47.7% in the RTX2080Ti platform and 51.2% in the DSPU platform.

VI. RELATED WORKS

A. Accelerators for 3D Spatial Computing System

With the development of deep learning technologies, many studies of 3D spatial computing accelerators have provided real-time and outstanding performance in limited battery capacity. For example, monocular depth estimation accelerators [20], [21] have been developed in ASICs that generate a high-quality 3D depth map from an RGB image through a DNN model, overcoming inherent problems of 3D scanners [53]. Although they only deploy single-camera applications, cooperating with CamPU is able to extend their functionality to multi-camera applications in minimal area overheads. Point cloud-based neural network accelerators [14], [19], [31], [51] have been explored in hardware architectures to achieve real-time 3D perception, such as 3D bounding box and semantic segmentation, by executing DNN models on 3D point cloud data. However, capturing wide FoV of reliable point cloud data is challenging in battery-limited edge devices. The CamPU-integrated DNN platform can capture a wide FoV 3D point cloud from multi-camera in low power through a multi-camera RGB-D capturing system [26], [30], [37], [41], [49]. Therefore, the CamPU-integrated DNN platform along with point cloud-based neural network accelerators can achieve high-speed and low-power wide FoV 3D point cloud applications in battery-limited edge devices not relying on heavy 3D scanners (e.g. LiDAR sensors).

Even in non-DNN applications, traditional 3D spatial computing systems exploit a multi-camera system. In specific, VR rendering works [28], [54] have focused on projecting a pre-synthesized omnidirectional spherical image onto a VR display. Their VR system targets real-time computations of the equation 1 regarding changing the user's head orientation as Stage 2 in Figure 2. Their works are orthogonal to CamPU which applies LUT-based image projection on a fixed multi-camera rig where Stage 2 is a minor operation. Since CamPU could build a 3D omnidirectional RGB-D map in runtime, the CamPU-integrated VR system along with a VR rendering accelerator could not only generate a 3D reconstruction map but also render it on a VR display in real-time. Recently, Simultaneous Localization and Mapping (SLAM) works [15], [24], [50] have exploited a multi-camera for high localization accuracy. However, their cameras have to be physically aligned with each other, and a tightly constraint chassis would fail feature matching between adjacent camera images [27]. CamPU could provide virtual camera images of any views through Stage 1 and Stage 2 executions. Therefore, SLAM accelerators integrating CamPU would support feasible multi-camera SLAM systems without strict constraints.

Many 3D spatial computing systems have required a multi-camera system to understand wide FoV scenes. CamPU is compatible with previous 3D spatial computing platforms and provides immersive information to them. As a result, CamPU can be deployed in any DNN or non-DNN spatial computing applications by cooperating with previous platforms, achieving

real-time and low-power multi-camera systems with minimal area overheads.

B. Acceleration of Image Stitching Algorithms

Image stitching is a fusion of images to generate a wide FoV image. An image stitching algorithm consists of feature extraction, feature matching, image projection, and image blending. Previous works [1], [23], [45] have focused on accelerating feature extraction [6], [32] and feature matching in unknown camera environments. Their methods are orthogonal to the CamPU platform which targets known camera setups in a multi-camera rig and does not need computing complex feature extraction and matching for image stitching. Another work [25] accelerates the LUT-based image projection to a fisheye image on a GPU platform. However, its single-image projection method cannot be extended to multi-camera image projection and blending that bring about critical issues in the GPU platforms as explained in Section II. Nowadays, commercial 360° cameras [22], [29] generate 360° images in real-time image stitching on more than two fisheye images. However, they cannot support Stage 2 (multi-image projection), Stage 3 (DNN), and Stage 4 (arbitrarily positioned multi-image stitching) in Figure 2 which are the inevitable processes in multi-camera deep learning-based spatial computing systems. Therefore, end-to-end system optimization is necessary for real-time and low-power implementations, and the CamPU-integrated DNN platform successfully demonstrates low-latency end-to-end 3D spatial computing systems.

VII. DISCUSSION

As commercial 360° cameras [22], [29] have developed, new DNN models [44], [48], [55] that trained directly on a 360° image have been studied. They modified conventional DNN models to distortion models aware of a 360° image. However, these algorithms are only compatible with full 360° image systems and cannot be used for tasks that require sub-FoV angles. For example, AR/VR rendering systems require 135° (Horizontal)×180° (Vertical) FoV images to fit in human eyes, which cannot perform the distortion DNN models trained directly on full 360° images. Moreover, acquiring reliable full 360° datasets is impractical. Specifically, a bulky RGB-D camera setup [8] captures a 360° RGB-D scene by rotating around all perspective views in a time series. The limited number of datasets loses generalization to unseen data [37], resulting in accuracy losses. The flexible CamPU system can generate any FoV angle by fusing tangent images, which is compatible with conventional DNN models pre-trained with sufficient rectilinear image datasets. As a result, the CamPU system does not rely on 360° cameras and 360° datasets, performing practical 3D spatial computing systems.

A limitation of this work could be DNN processing when the number of cameras increases dramatically. Some studies [2], [30] exploit more than 26 cameras for the highest accuracy. However, a DNN accelerator cannot handle that amount of tangent images with its limited resources, limiting DNN performance enhancement during batch processing. To

alleviate the overheads of numerous DNN executions, the CampPU-integrated DNN platform could adopt efficient DNN optimization methods such as spatial-temporal DNN computational reuse [10], [33], [34], [43], [52]. Consequently, the system optimizations between CampPU and DNN accelerators would be an interesting research topic for future proposals.

VIII. CONCLUSION

CampPU is designed for deploying a multi-camera system in 3D spatial computing applications. The previous GPU platform slows down a multi-camera system as the number of cameras increases, unlike multi-batch DNN processing. On the other hand, CampPU improves the performance of a multi-camera system by exploiting shape similarity (inter-data reuse) and value similarity of mapping indices (intra-data reuse). Moreover, the out-of-order image projection unit with cache memory is proposed to reduce the number of cache memory accesses and hide the latency of high-level memory accesses. Additionally, the overlap-aware blending unit handles overlapping regions between adjacent images during image blending, minimizing redundant memory footprints. The architecture and system evaluations are presented to provide a comprehensive analysis of CampPU. Finally, the CampPU-integrated DNN platform achieves 94.1 ms of latency for multi-camera deep learning-based 360° RGB-D generation.

REFERENCES

- [1] K. Abughalieh, O. Bataineh, and S. Alawneh, "Acceleration of image stitching using embedded graphics processing unit," in *2018 IEEE International Conference on Electro/Information Technology (EIT)*. IEEE, 2018, pp. 0035–0039.
- [2] H. Ai, Z. Cao, Y.-P. Cao, Y. Shan, and L. Wang, "Hrdfs: Monocular 360deg depth estimation by collaboratively learning holistic-with-regional depth distributions," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 13 273–13 282.
- [3] Apple, "Apple Vision Pro," Apple Inc. [Online]. Available: <https://www.apple.com/apple-vision-pro/specs/>
- [4] Apple Developer, "Capturing Photos with Depth," Apple Inc., Inc.
- [5] Ashok Elluswamy, "Foundation Models for Autonomy," *The IEEE / CVF Computer Vision and Pattern Recognition Conference (CVPR) Workshop on Autonomous Driving*, 2023.
- [6] H. Bay, T. Tuytelaars, and L. Van Gool, "Surf: Speeded up robust features," in *Computer Vision—ECCV 2006: 9th European Conference on Computer Vision, Graz, Austria, May 7–13, 2006. Proceedings, Part I* 9. Springer, 2006, pp. 404–417.
- [7] J. Bian, Z. Li, N. Wang, H. Zhan, C. Shen, M.-M. Cheng, and I. Reid, "Unsupervised scale-consistent depth and ego-motion learning from monocular video," *Advances in neural information processing systems*, vol. 32, 2019.
- [8] A. Chang, A. Dai, T. Funkhouser, M. Halber, M. Niessner, M. Savva, S. Song, A. Zeng, and Y. Zhang, "Matterport3d: Learning from rgb-d data in indoor environments," *arXiv preprint arXiv:1709.06158*, 2017.
- [9] G. Chaurasia, A. Nieuwoudt, A.-E. Ichim, R. Szeliski, and A. Sorkine-Hornung, "Passthrough+ real-time stereoscopic view synthesis for mobile mixed reality," *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, vol. 3, no. 1, pp. 1–17, 2020.
- [10] N. M. Cicek, L. Ning, O. Ozturk, and X. Shen, "General reuse-centric cnn accelerator," *IEEE Transactions on Computers*, vol. 71, no. 4, pp. 880–891, 2021.
- [11] Cypress Semiconductor, "512 Mb (64 MB)/256 Mb (32 MB)/ 128 Mb (16 MB) HYPERFLASH family Data-Sheet," Cypress Int., Inc.
- [12] Cypress Semiconductor, "64Mbit HyperRAM Self-Refresh DRAM Data-Sheet," Cypress Int., Inc.
- [13] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [14] Y. Feng, B. Tian, T. Xu, P. Whatmough, and Y. Zhu, "Mesorasi: Architecture support for point cloud analytics via delayed-aggregation," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 1037–1050.
- [15] Y. Gan, Y. Bo, B. Tian, L. Xu, W. Hu, S. Liu, Q. Liu, Y. Zhang, J. Tang, and Y. Zhu, "Eudoxus: Characterizing and accelerating localization in autonomous machines industry track paper," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 827–840.
- [16] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the kitti vision benchmark suite," in *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 2012, pp. 3354–3361.
- [17] C. Godard, O. Mac Aodha, M. Firman, and G. J. Brostow, "Digging into self-supervised monocular depth estimation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 3828–3838.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [19] D. Im, D. Han, S. Kang, and H.-J. Yoo, "A pipelined point cloud based neural network processor for 3-d vision with large-scale max pooling layer prediction," *IEEE Journal of Solid-State Circuits*, vol. 57, no. 2, pp. 661–670, 2021.
- [20] D. Im, G. Park, Z. Li, J. Ryu, S. Kang, D. Han, J. Lee, W. Park, H. Kwon, and H.-J. Yoo, "A mobile 3d object recognition processor with deep learning-based monocular depth estimation," *IEEE Micro*, 2023.
- [21] D. Im, G. Park, Z. Li, J. Ryu, S. Kang, D. Han, J. Lee, and H.-J. Yoo, "Dspu: A 281.6 mw real-time depth signal processing unit for deep learning-based dense rgb-d data acquisition with depth fusion and 3d bounding box extraction in mobile platforms," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 510–512.
- [22] Insta360, "Insta360 X3," Insta360 Inc. [Online]. Available: <https://onlinemanual.insta360.com/x3/en-us/faq/specs>
- [23] Y. Jia, R. Wang, and X. Jiang, "A real-time image stitching and fusion algorithm circuit design based on fpga," *Electronics*, vol. 13, no. 2, p. 271, 2024.
- [24] P. Kaveti, S. N. Vaidyanathan, A. T. Chelvan, and H. Singh, "Design and evaluation of a generic visual slam framework for multi camera systems," *IEEE Robotics and Automation Letters*, 2023.
- [25] Y.-W. Kim, H.-S. Yang, and D. Kim, "Real-time lens distortion algorithm on an edge device with gpu," *IEEE Access*, vol. 10, pp. 41 748–41 757, 2022.
- [26] R. Komatsu, H. Fujii, Y. Tamura, A. Yamashita, and H. Asama, "Free viewpoint image generation system using fisheye cameras and a laser rangefinder for indoor robot teleoperation," *ROBOMECH Journal*, vol. 7, pp. 1–10, 2020.
- [27] J. Kuo, M. Muglikar, Z. Zhang, and D. Scaramuzza, "Redesigning slam for arbitrary multi-camera systems," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 2116–2122.
- [28] Y. Leng, C.-C. Chen, Q. Sun, J. Huang, and Y. Zhu, "Energy-efficient video processing for virtual reality," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 91–103.
- [29] LG, "LG 360 CAM," LG Inc. [Online]. Available: https://www.lg.com/ca_en/cell-phones/lgr105/
- [30] Y. Li, Y. Guo, Z. Yan, X. Huang, Y. Duan, and L. Ren, "Omnifusion: 360 monocular depth estimation via geometry-aware fusion," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 2801–2810.
- [31] Y. Lin, Z. Zhang, H. Tang, H. Wang, and S. Han, "Pointacc: Efficient point cloud accelerator," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 449–461.
- [32] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International journal of computer vision*, vol. 60, pp. 91–110, 2004.
- [33] M. Mahmoud, K. Siu, and A. Moshovos, "Diffy: A déjà vu-free differential deep neural network accelerator," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 134–147.

- [34] L. Ning and X. Shen, "Deep reuse: Streamline cnn inference on the fly via coarse-grained computation reuse," in *Proceedings of the ACM International Conference on Supercomputing*, 2019, pp. 438–448.
- [35] NVIDIA, "NVIDIA GeForce RTX 2080 Ti Graphics Card." [Online]. Available: <https://www.nvidia.com/en-us/geforce/20-series/>
- [36] C. R. Qi, O. Litany, K. He, and L. J. Guibas, "Deep hough voting for 3d object detection in point clouds," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 9277–9286.
- [37] M. Rey-Area, M. Yuan, and C. Richardt, "360monodepth: High-resolution 360deg monocular depth estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 3762–3772.
- [38] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III 18*. Springer, 2015, pp. 234–241.
- [39] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. S. Bernstein, A. C. Berg, and L. Fei-Fei, "Imagenet large scale visual recognition challenge," *CoRR*, vol. abs/1409.0575, 2014. [Online]. Available: <http://arxiv.org/abs/1409.0575>
- [40] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.
- [41] Y. Shi, H. Cai, A. Ansari, and F. Porikli, "Ega-depth: Efficient guided attention for self-supervised multi-camera depth estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 119–129.
- [42] J. P. Snyder, *Map projections—A working manual*. US Government Printing Office, 1987, vol. 1395.
- [43] Z. Song, F. Wu, X. Liu, J. Ke, N. Jing, and X. Liang, "Vr-dann: Real-time video recognition via decoder-assisted neural network acceleration," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 698–710.
- [44] C. Sun, M. Sun, and H.-T. Chen, "Hohonet: 360 indoor holistic understanding with latent horizontal features," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 2573–2582.
- [45] S. W. Tesfay, Z. G. Demirdag, H. F. Ugurdag, and H. F. Ates, "Hybrid cpu-gpu acceleration of a multithreaded image stitching algorithm," in *2022 7th International Conference on Computer Science and Engineering (UBMK)*. IEEE, 2022, pp. 468–473.
- [46] D. Ungureanu, F. Bogo, S. Galliani, P. Sama, X. Duan, C. Meekhof, J. Stühmer, T. J. Cashman, B. Tekin, J. L. Schönberger *et al.*, "Hololens 2 research mode as a tool for computer vision research," *arXiv preprint arXiv:2008.11239*, 2020.
- [47] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [48] F.-E. Wang, Y.-H. Yeh, M. Sun, W.-C. Chiu, and Y.-H. Tsai, "Bifuse: Monocular 360 depth estimation via bi-projection fusion," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 462–471.
- [49] S. Xie, D. Wang, and Y.-H. Liu, "Omnividar: omnidirectional depth estimation from multi-fisheye images," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 21 529–21 538.
- [50] Y. Yang, M. Pan, D. Tang, T. Wang, Y. Yue, T. Liu, and M. Fu, "Mcovslam: A multicamera omnidirectional visual slam system," *IEEE/ASME Transactions on Mechatronics*, 2024.
- [51] Z. Ying, S. Bhuyan, Y. Kang, Y. Zhang, M. T. Kandemir, and C. R. Das, "Edgepc: Efficient deep learning analytics for point clouds on edge devices," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.
- [52] Z. Yuan, Y. Yang, J. Yue, R. Liu, X. Feng, Z. Lin, X. Wu, X. Li, H. Yang, and Y. Liu, "14.2 a 65nm 24.7 μ j/frame 12.3 mw activation-similarity-aware convolutional neural network video processor using hybrid precision, inter-frame data reuse and mixed-bit-width difference-frame data codec," in *2020 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 2020, pp. 232–234.
- [53] Y. Zhang and T. Funkhouser, "Deep depth completion of a single rgb-d image," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 175–185.
- [54] S. Zhao, H. Zhang, S. Bhuyan, C. S. Mishra, Z. Ying, M. T. Kandemir, A. Sivasubramaniam, and C. R. Das, "Déja view: Spatio-temporal compute reuse for 'energy-efficient 360 vr video streaming,'" in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 241–253.
- [55] N. Zioulis, A. Karakottas, D. Zarpalas, and P. Daras, "OmniDepth: Dense depth estimation for indoors spherical panoramas," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 448–465.